

Instituto Tecnológico Autónomo de México

Cómputo Paralelo y en la Nube

Evaluación experimental de desempeño

DBSCAN paralelo para la detección de ruido o outliers con OpenMP

Curso	Cómputo Paralelo y en la Nube
Profesor	Dr. J. Octavio Gutiérrez García
Equipo	Marcio Tellez e Irene Escudero
Entrega	20 de octubre de 2026
Repositorio	github.com/TellezM/Proyecto_Par_y_Nube

1. Definición del experimento

Queremos saber cuánto más rápidas son las dos versiones paralelas (P1: matriz completa, P2: matriz dividida en `k` regiones) que la versión serial, y cómo cambia eso con el número de regiones, el número de puntos y el número de hilos. El código y las decisiones de diseño están en el otro documento de la entrega; aquí se describe y se analiza el experimento.

1.1 Parámetros

Tabla 1. *Parámetros del experimento.*

Parámetro	Valores
Versiones	serial, P1, P2
k (regiones, solo P2)	2, 4, 8, 16, 32, 64
Número de puntos	20,000; 40,000; 80,000; 120,000; 140,000; 160,000; 180,000; 200,000
Número de hilos	{1, cores/2, cores, cores×2} con cores = 2 núcleos: {1, 1, 2, 4}, es decir, 1, 2 y 4 (cores/2 coincide con 1)
Iteraciones	10 por configuración (se promedian)
Parámetros de DBSCAN	`eps = 0.03`, `min_samples = 10` (los de la libreta del curso)
Banderas de compilación	`g++ -O2 -fopenmp`

La versión serial se mide una vez por cada tamaño (no usa hilos), P1 con 3 números de hilos y P2 con 6 valores de `k` y 3 números de hilos. Con los 8 tamaños son 176 configuraciones; con 10 iteraciones cada una, 1,760 corridas. El archivo de datos tiene 1,760 filas, una por corrida.

1.2 Entrada y salida

La entrada son puntos aleatorios generados con la misma receta de `DBSCAN_noise.ipynb` (`make_blobs` con 4 centros, `cluster_std = 0.06`, `random_state = 11`, valores positivos y tres decimales), para cada uno de los 8 tamaños. Todas las versiones y todas las iteraciones usan exactamente el mismo archivo para un mismo tamaño. La salida es un CSV con `x,y,etiqueta` (0 ruido, 1 core de primer orden, 2 core de segundo orden), que se puede graficar con la libreta.

1.3 Qué se mide y cómo se compara

El tiempo es el del algoritmo, medido dentro del programa con `omp_get_wtime()`. No incluye leer ni escribir los CSV, y sí incluye lo que P2 necesita para dividir la matriz (reordenar los puntos y armar la cuadrícula). Para cada configuración se promedian las iteraciones. El speedup se calcula contra la versión serial con la misma entrada:

```
speedup(config, N)  =  tiempo_medio_serial(N) / tiempo_medio(config, N)
```

Para entender de dónde viene la mejora de P2 se separan dos efectos. La **ganancia algorítmica** es el tiempo de la serial entre el de P2 con 1 hilo: lo que se gana solo por dividir en regiones. La

ganancia paralela es el tiempo de la misma versión con 1 hilo entre el tiempo con varios hilos, y la **eficiencia** es esa ganancia dividida entre el número de hilos (1.0 sería un reparto perfecto).

Las corridas se hicieron en rondas: en cada ronda se ejecutó una iteración de todas las configuraciones y de todos los tamaños, para que cualquier variación del sistema durante las horas que dura el experimento se reparta parejo entre versiones y no favorezca a ninguna. Además, en cada una de las 1,760 corridas se comparó la salida contra la de la versión serial (columna `salida_ok` del archivo de datos): todas coinciden.

2. Equipo donde se ejecutaron los experimentos

Los experimentos se corrieron en un equipo Linux con 2 núcleos, por lo que el número de cores es 2 en este experimento.

Tabla 2. *Hardware y software.*

	Característica
Procesador	Intel(R) Xeon(R) Processor @ 2.80GHz (arquitectura x86_64, familia 6, modelo 85)
Núcleos	2 (1 hilo por núcleo, 1 socket)
Caché	L1d 64 KiB (2 instances); L2 2 MiB (2 instances); L3 33 MiB (1 instance)
Memoria RAM	8 GB
Sistema operativo	Ubuntu 24.04.5 LTS
Compilador	g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Compilación	`g++ -O2 -fopenmp proy_ruido.cpp -o dbscan` (OpenMP de GCC)
Análisis	Python 3, pandas, matplotlib, scikit-learn (solo para generar datos y validar)

Con solo dos núcleos no se puede observar cómo escala el programa con 8 o 16 hilos; con 4 hilos (el doble de los núcleos, como pide la rúbrica) hay más hilos que núcleos. Hay variación entre corridas, por eso se promedian 10 iteraciones y se reporta la desviación estándar.

3. Resultados

3.1 Tiempos y speedups con la entrada más grande

Tabla 3. *Resultados con 200,000 puntos (promedio de 10 iteraciones).*

Versión	k	Hilos	Tiempo medio (s)	Desv. est. (s)	Speedup	Eficiencia
Serial	–	1	40.63	0.78	1.00	–
P1	–	1	40.78	0.60	1.00	1.00
P1	–	2	20.43	0.21	1.99	1.00

Versión	k	Hilos	Tiempo medio (s)	Desv. est. (s)	Speedup	Eficiencia
P1	–	4	20.49	0.37	1.98	0.50
P2	2	1	40.79	0.70	1.00	1.00
P2	2	2	21.21	0.51	1.92	0.96
P2	2	4	21.19	0.45	1.92	0.48
P2	4	1	40.37	0.69	1.01	1.00
P2	4	2	24.66	0.32	1.65	0.82
P2	4	4	21.30	0.25	1.91	0.47
P2	8	1	28.43	0.57	1.43	1.00
P2	8	2	16.83	0.33	2.41	0.84
P2	8	4	16.85	0.15	2.41	0.42
P2	16	1	18.90	0.40	2.15	1.00
P2	16	2	9.90	0.18	4.10	0.95
P2	16	4	10.63	0.21	3.82	0.44
P2	32	1	11.70	0.26	3.47	1.00
P2	32	2	6.02	0.15	6.75	0.97
P2	32	4	6.12	0.11	6.64	0.48
P2	64	1	9.90	0.19	4.11	1.00
P2	64	2	5.14	0.14	7.91	0.96
P2	64	4	5.34	0.26	7.61	0.46

3.2 Speedup según el número de hilos, k y número de puntos

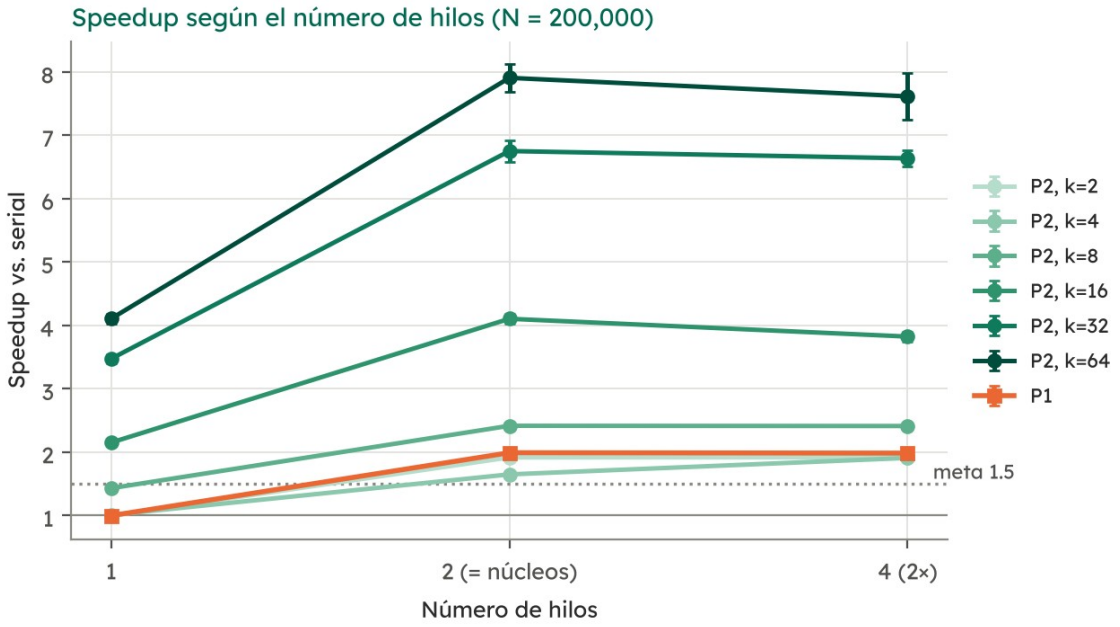


Figura 1. *Speedup según el número de hilos con 200,000 puntos. Las barras de error son la desviación estándar de las 10 iteraciones.*

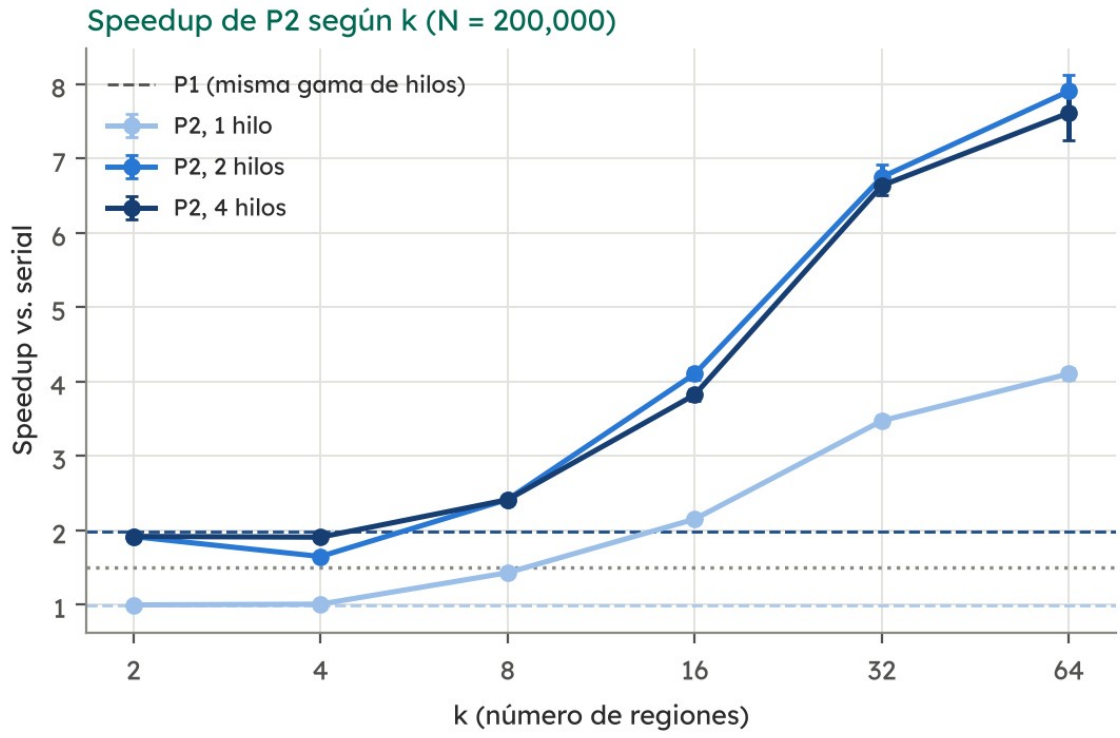


Figura 2. *Speedup de P2 según k con 200,000 puntos, una curva por número de hilos. Las líneas punteadas son P1 con el mismo número de hilos.*

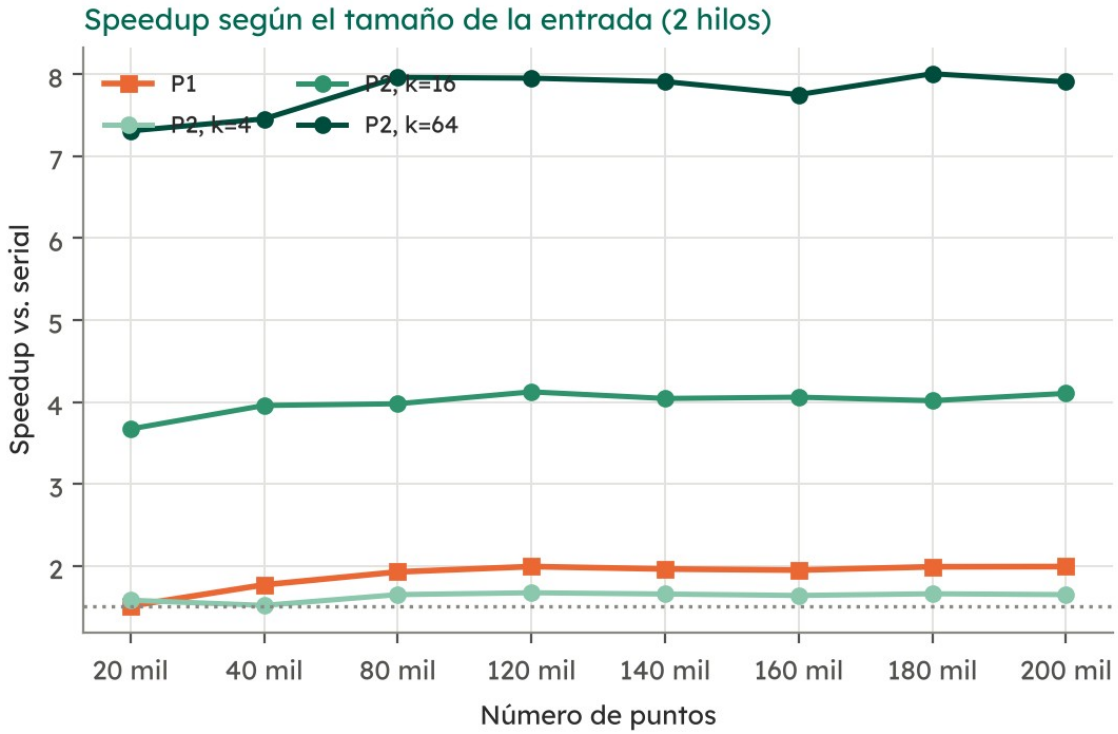


Figura 3. *Speedup según el número de puntos con 2 hilos (el número de núcleos).*

Tabla 4. *Speedup con 2 hilos para todos los tamaños.*

	20 mil	40 mil	80 mil	120 mil	140 mil	160 mil	180 mil	200 mil
P1	1.51	1.77	1.92	1.99	1.96	1.95	1.99	1.99
P2, k=2	1.65	1.71	1.91	1.99	1.94	1.88	1.94	1.92
P2, k=4	1.58	1.52	1.65	1.67	1.65	1.64	1.66	1.65
P2, k=8	2.37	2.53	2.44	2.63	2.44	2.44	2.42	2.41
P2, k=16	3.67	3.96	3.98	4.12	4.04	4.06	4.02	4.10
P2, k=32	6.16	6.48	6.48	6.63	6.58	6.73	6.73	6.75
P2, k=64	7.31	7.46	7.96	7.95	7.91	7.75	8.01	7.91

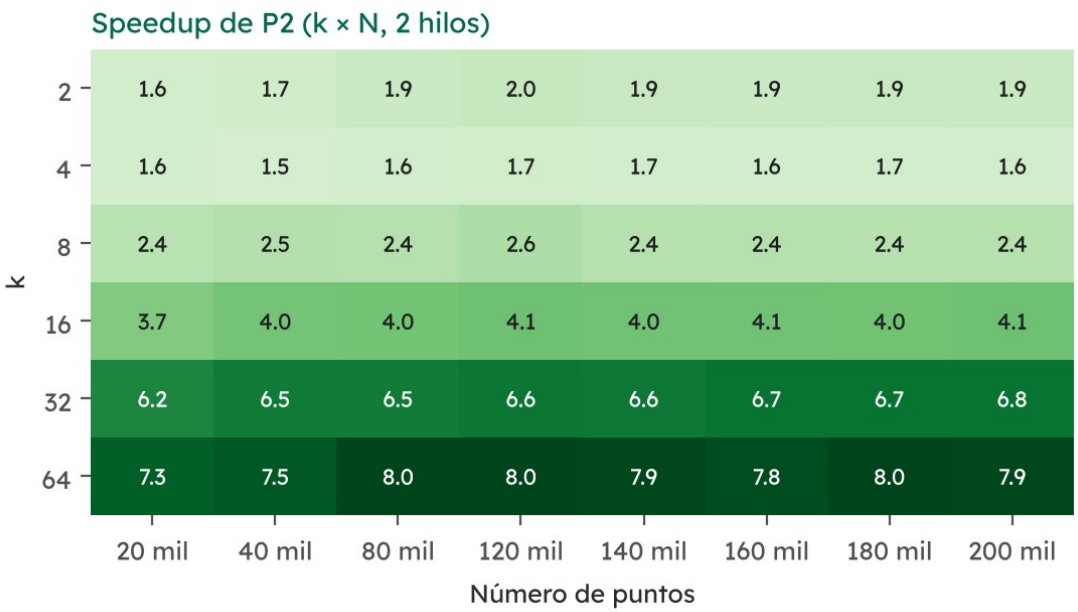


Figura 4. *Speedup de P2 con 2 hilos según k y número de puntos.*

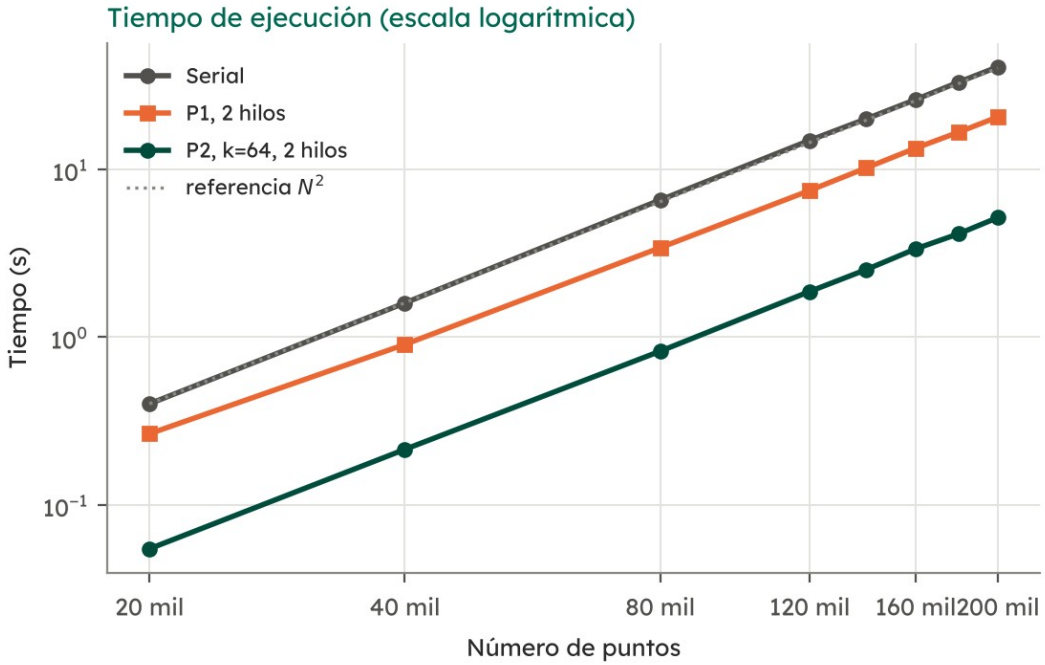


Figura 5. *Tiempos de ejecución en escala logarítmica. La referencia punteada crece como N^2.*

3.3 De dónde viene la mejora y qué tan bien se reparte el trabajo

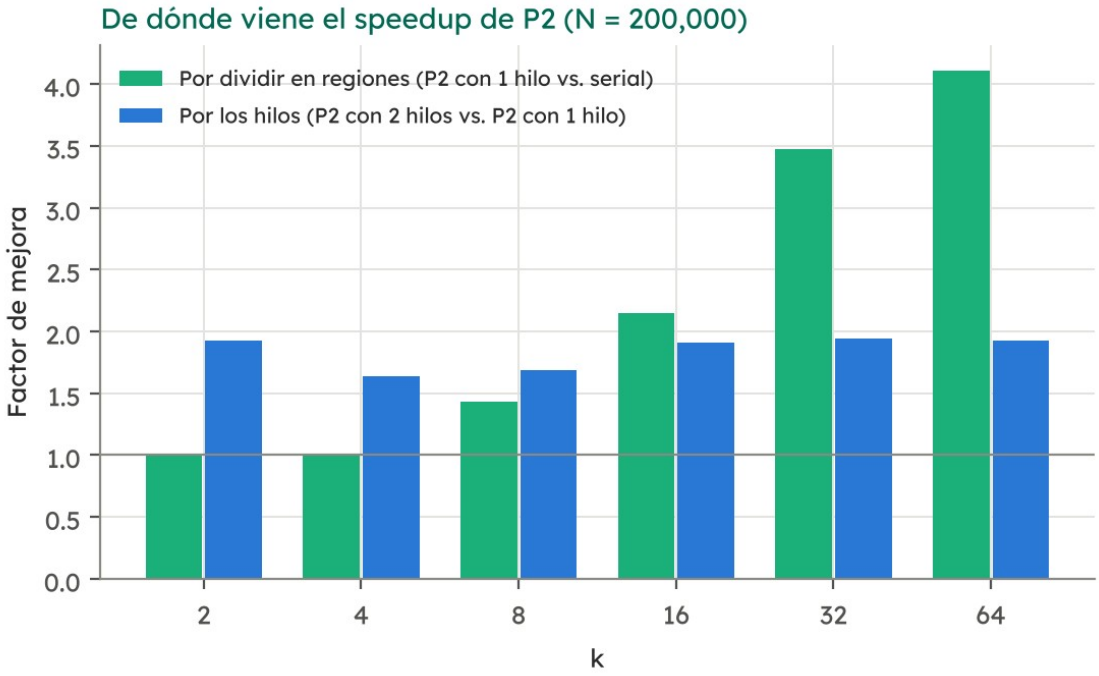


Figura 6. *Ganancia por dividir en regiones (P2 con 1 hilo contra la serial) y ganancia por usar 2 hilos (P2 con 2 hilos contra P2 con 1 hilo), con 200,000 puntos.*

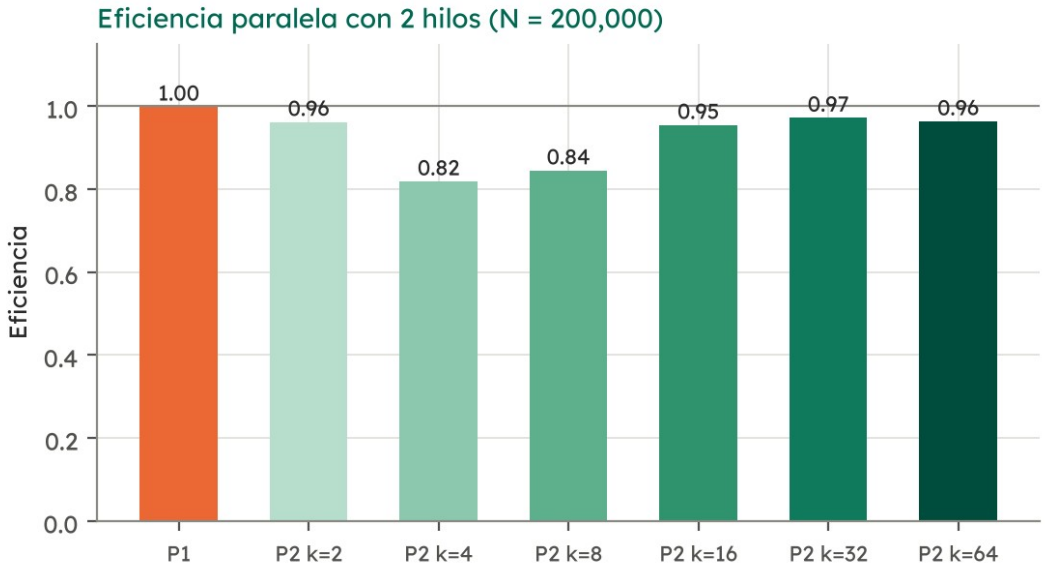


Figura 7. *Eficiencia paralela con 2 hilos y 200,000 puntos.*

4. Análisis e interpretación

4.1 ¿Se cumple la meta de speedup mayor a 1.5?

Sí. El mejor speedup de P1 fue 1.99 (con 4 hilos y 120,000 puntos) y el de P2 fue 8.01 (k=64, 2 hilos, 180,000 puntos). Los dos superan 1.5, así que se cumple la condición de la rúbrica con cualquiera de las dos versiones paralelas. Con la entrada más grande y 2 hilos, P1 llega a 1.99 y P2 va desde 1.92 con k=2 hasta 7.91 con k=64.

4.2 Versión P1: matriz completa

P1 hace exactamente el mismo trabajo que la serial, repartido entre hilos, así que su límite es el número de núcleos. Con 2 hilos llega a 1.99, una eficiencia de 1.00: casi un reparto perfecto entre los dos núcleos. Con 1 hilo da 1.00, es decir, la misma velocidad que la serial (la parte paralela no agrega costo apreciable). Con 4 hilos da 1.98: no hay más núcleos, así que los hilos adicionales se turnan en los mismos dos núcleos y no mejoran (ni empeoran de forma importante, porque el trabajo está bien repartido y no hay sincronización). Su rendimiento casi no cambia con el tamaño de la entrada (entre 1.51 y 1.99 con 2 hilos), que es lo esperado en un algoritmo cuyas iteraciones son todas iguales.

4.3 Versión P2: matriz dividida en k regiones

El rendimiento de P2 depende mucho de k. Con 2 hilos y 200,000 puntos el speedup va así: k=2: 1.92, k=4: 1.65, k=8: 2.41, k=16: 4.10, k=32: 6.75, k=64: 7.91. Para entender por qué hay que separar los dos efectos de la sección 3 (Figura 6).

El efecto de algoritmo es el de dividir en regiones: con muchas regiones cada punto se compara solo con los de su región y su halo, que son una fracción pequeña del total. La ganancia algorítmica (P2 con 1 hilo contra la serial, donde 1 significa que no hay ganancia) es de 1.00 con k=2 y de 4.11 con k=64. Con pocas regiones las celdas son enormes (con k=2 cada una cubre la mitad del plano), el halo cubre casi todo y casi no se descartan comparaciones; con k=64 cada celda es una parte pequeña del plano y se descarta la mayor parte de las comparaciones.

El efecto de paralelismo es el de repartir las regiones entre hilos. Con 2 hilos la ganancia paralela de P2 va de 1.64 a 1.94 según k, y la eficiencia no es la misma para todos los valores: con k=4 y k=8 es claramente menor. La razón es que las regiones no tienen el mismo número de puntos (hay cuatro grupos densos en el plano) y P2 reparte regiones completas, así que un hilo puede quedarse con mucho más trabajo que el otro. Para comprobarlo contamos los puntos de cada región con la entrada de 200,000 puntos, calculamos el trabajo de cada una (puntos de la región por puntos de su región y su halo, que es el número de comparaciones de la fase 1) y simulamos cómo se reparten esas cargas entre 2 hilos con `schedule(dynamic, 1)`. La Tabla 5 compara la mejor eficiencia que permite ese reparto con la que medimos.

Tabla 5. *Balance de carga de las regiones de P2 con 200,000 puntos y 2 hilos.*

k	Cuadrícula	Puntos por región (mín – máx)	Trabajo de la región más pesada / trabajo medio	Eficiencia máxima del reparto (modelo)	Eficiencia medida (2 hilos)
2	1x2	97,092 – 102,908	1.03	0.97	0.96
4	2x2	23,667 – 79,241	1.58	0.83	0.82
8	2x4	116 – 49,876	2.06	0.85	0.84
16	4x4	0 – 42,795	3.69	0.95	0.95
32	4x8	0 – 25,005	4.55	1.00	0.97
64	8x8	0 – 21,076	6.69	0.99	0.96

Los dos números coinciden bastante bien: con k=4 las regiones tienen entre 23,667 y 79,241 puntos y el reparto no puede pasar de 0.83; con k=8 el límite es 0.85. Con k=32 y k=64 hay tantas regiones, y tan pequeñas, que el reparto es casi perfecto. Esto explica por qué conviene usar más regiones que hilos. La eficiencia de P1 (1.00) es mejor que la de P2 con pocas regiones porque en P1 todas las iteraciones cuestan lo mismo.

4.4 Por qué una versión paralela fue mejor que la otra

Depende de k. P2 supera a P1 a partir de k=8 (con 2 hilos y 200,000 puntos); con k menores es equivalente o peor. Cuando P2 gana, no es porque paralelice mejor, sino porque hace mucho menos trabajo: la ganancia algorítmica de P2 es la que domina (Figura 6), mientras que P1 paraleliza mejor el mismo trabajo (su eficiencia es más alta). Dicho de otra forma, P1 es una versión paralela "honesta" del algoritmo ingenuo, con speedup limitado por los núcleos; P2 es una versión que cambia la cantidad de comparaciones y por eso puede superar el número de núcleos (por ejemplo, 7.91 con k=64 en una máquina de 2 núcleos). Si solo se cuenta lo que se debe a los hilos, ninguna de las dos supera los 2 núcleos.

Hay que decir con claridad qué mide el speedup de P2 contra la serial: la comparación mezcla la ventaja de usar una cuadrícula con la de usar hilos. Por eso mostramos las dos ganancias por separado, y por eso la conclusión sobre cuál versión es "mejor" depende de lo que se quiera comparar.

4.5 Efecto del número de puntos

Los tiempos crecen como N^2 en las tres versiones (Figura 5), porque cada punto se compara contra una fracción del total que no cambia con N. El speedup de P1 es casi constante. El de P2 con k=64 va de 7.31 a 8.01 según el tamaño con 2 hilos; con k=2, de 1.65 a 1.99. En una primera versión de P2 (con listas de índices) el speedup caía con el tamaño; guardar los puntos reordenados por región (ver el otro documento, sección 6.5) corrigió gran parte de ese efecto.

4.6 Efecto del número de hilos

Pasar de 1 a 2 hilos mejora todas las versiones, cerca del doble cuando el trabajo se reparte bien. Pasar de 2 a 4 hilos no mejora, porque la máquina tiene 2 núcleos: con k=64, P2 da 4.11,

7.91 y 7.61 con 1, 2 y 4 hilos. Es el comportamiento que se espera de la sobresuscripción: más hilos que núcleos no crean más capacidad de cómputo.

4.7 Variación entre corridas

La desviación estándar relativa de las 10 iteraciones fue de 3.1% en la mediana de las configuraciones y de 25.0% en el peor caso. Algunas corridas aisladas pueden salir más lentas por otras tareas del sistema. Diferencias entre configuraciones menores a esa variación no deben interpretarse.

4.8 Limitaciones

- Solo hay 2 núcleos: no se puede evaluar cómo escalan P1 y P2 con 8, 16 o más hilos. Con más núcleos esperaríamos que el speedup de P1 siguiera creciendo hasta cerca del número de núcleos y que el de P2 quedara limitado por el balance de las regiones (por eso conviene usar más regiones que hilos).
- Los datos tienen cuatro grupos densos con el mismo tamaño. P2 se beneficia de ello porque descarta regiones vacías o lejanas. Con puntos uniformes, o con un grupo mucho más grande que los demás, los resultados serían distintos.
- Todos los experimentos usan `-O2`. Con `-O3 -march=native` todas las versiones mejoran entre 2 y 3 veces y el speedup de P2 contra la serial casi no cambia (ver el otro documento, sección 6.7), pero no se repitió el experimento completo con esas banderas.
- `eps` y `min_samples` se mantuvieron fijos en 0.03 y 10, como en la libreta del curso.

5. Conclusiones

- Las dos versiones paralelas cumplen la condición de speedup mayor a 1.5: P1 llega a 1.99 y P2 a 8.01.
- P1 aprovecha casi por completo los núcleos disponibles (1.99 con 2 hilos), pero no puede pasar de ahí porque hace todo el trabajo del algoritmo ingenuo.
- P2 depende de `k`: con pocas regiones es equivalente o peor que P1 y con muchas regiones es varias veces mejor, sobre todo porque evita comparaciones y no tanto por usar más hilos.
- Los hilos por encima del número de núcleos no ayudan.
- Las decisiones de diseño (qué ciclo paralelizar, cómo planificar, cómo guardar los puntos de cada región) se tomaron con mediciones y están documentadas en el otro documento.

6. Uso responsable y ético de IA generativa

Seguimos los lineamientos de IEEE y de Elsevier sobre IA generativa, que piden declarar qué herramienta se usó y para qué, que la IA no se considere autora y que las personas asuman la responsabilidad total del contenido. Lo declaramos así.

Herramienta. Usamos Claude, un modelo de IA generativa de Anthropic, en una sesión de trabajo el 19 de octubre de 2026, con acceso a una carpeta del proyecto y a un equipo Linux donde compiló y corrió los programas.

Para qué se usó. Leyó las instrucciones y la rúbrica del proyecto y las diapositivas del curso; revisó el código que había escrito Marcio Tellez (versión serial, P1 y P2) y propuso correcciones, que son las que se describen en el otro documento: usar `double` en lugar de `float`, separar el arreglo `core` del arreglo de etiquetas para evitar lecturas y escrituras simultáneas, generalizar el halo de P2 y reordenar los puntos por región; escribió los programas de apoyo (generación de datos, pruebas de correctitud, estudio de decisión, experimento completo, análisis y gráficas); ejecutó las compilaciones y los experimentos en el equipo Linux de trabajo; y redactó el primer borrador de los dos documentos de la entrega.

Qué hicimos nosotros. Marcio escribió la primera versión del código. Los dos definimos el alcance con las instrucciones del proyecto, revisamos el código y las decisiones, y revisamos y ajustamos los documentos antes de entregarlos.

Cómo verificamos lo que produjo la IA. No tomamos nada solo por venir de la herramienta. El código se comparó contra la versión serial y contra `sklearn.DBSCAN` en distintos datos; en cada corrida del experimento se comparó la salida contra la serial; las decisiones de diseño se tomaron con mediciones y no con suposiciones; y todas las cifras de este documento salen del archivo de datos de los experimentos (`resultados_tiempos.csv`), sin estimaciones. La primera versión de P2 tenía un problema de rendimiento que solo se vio al medir (sección 6.5 del otro documento), lo que muestra por qué hay que verificar.

Límites y cuidados. Una herramienta de IA puede equivocarse o explicar con seguridad algo falso, así que lo que no se pudo comprobar con una prueba o con una medición se marcó como hipótesis (por ejemplo, la explicación de la caché). No se compartió con la herramienta información personal; solo el material del curso (diapositivas e instrucciones) y el código del proyecto.

Responsabilidad. La herramienta no es autora de este trabajo. Nosotros respondemos por el contenido y por entender el código: podemos explicar cada parte del programa y cada decisión que aquí se describe.

Anexo: capturas de la ejecución

Primero, el resultado de la detección de ruido sobre 4,000 puntos, comparado con `sklearn.DBSCAN` *(las mismas etiquetas de ruido). Después, la salida de la terminal donde se corrió todo, tal como la produjeron los programas.*

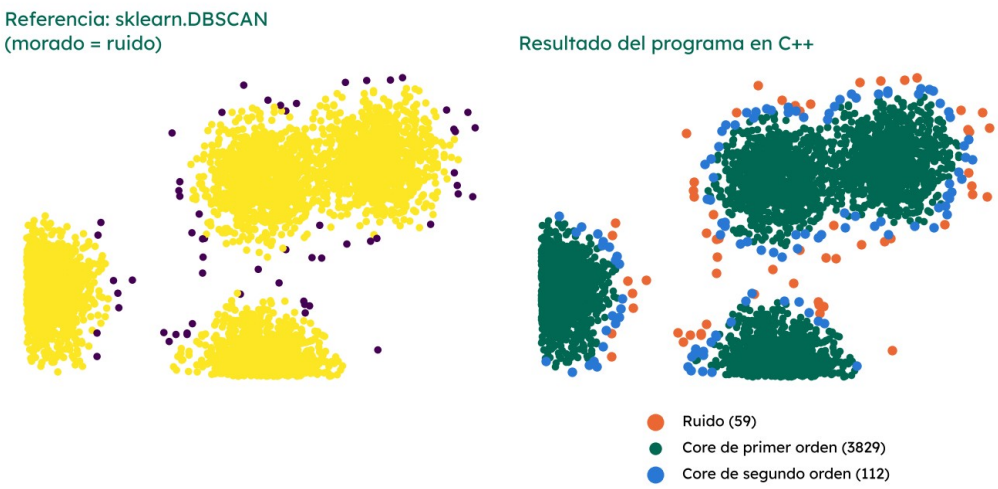


Figura 8. *Resultado de la detección de ruido sobre 4,000 puntos con P2 (k=16): izquierda sklearn, derecha el programa en C++.*

compilación

```
$ g++ --version | head -1
g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
$ g++ -O2 -fopenmp proy_ruido.cpp -o dbscan
(compilación sin errores ni advertencias)
$ ls -l dbscan
-rwxr-xr-x 44456 dbscan
```

Figura 9. *Compilación del programa y versión del compilador.*

ejecución

```
$ ./dbscan datos/4000_data.csv salida_serial.csv 0.03 10 serial 1 1
0.0153135
$ ./dbscan datos/4000_data.csv salida_p1.csv 0.03 10 p1 1 4
0.0120286
$ ./dbscan datos/4000_data.csv salida_p2.csv 0.03 10 p2 16 4
0.00396761
$ head -4 salida_p2.csv
0.415,0.446,1
0.47,0.007,1
0.784,0.5,1
0.406,0.451,1
$ cut -d, -f3 salida_p2.csv | sort | uniq -c
     59 0
   3829 1
    112 2
$ md5sum salida_serial.csv salida_p1.csv salida_p2.csv
d77638cd9e50...  salida_serial.csv
d77638cd9e50...  salida_p1.csv
d77638cd9e50...  salida_p2.csv
```

Figura 10. *Ejecución de las tres versiones sobre 4,000 puntos: tiempo impreso, primeras líneas del CSV de salida y conteo de etiquetas.*

correctitud

```
$ python3 scripts/prueba_correctitud.py ./dbscan
190/190 comparaciones correctas
```

Figura 11. *Pruebas de correctitud contra la versión serial y contra sklearn.*

experimento

```
$ tail -3 exp.log
ronda 9/10  [1584/1760]  14057s
ronda 10/10  [1760/1760]  15626s
LISTO 15626s
$ head -4 resultados_tiempos.csv
version,k,n_puntos,hilos,iteracion,tiempo_segundos,salida_ok
serial,0,20000,1,1,0.393125,1
p1,0,20000,1,1,0.377045,1
p1,0,20000,2,1,0.275576,1
$ wc -l resultados_tiempos.csv
1761 resultados_tiempos.csv
```

Figura 12. *Corrida del experimento completo: avance por rondas y primeras filas del archivo de resultados.*

análisis

```
$ python3 scripts/analizar_resultados.py resultados/resultados_tiempos.csv imagenes/experimento --cores
    2
N máx: 200000 | hilos: [np.int64(1), np.int64(2), np.int64(4)] | k: [np.int64(2), np.int64(4),
    np.int64(8), np.int64(16), np.int64(32), np.int64(64)]
Mejor speedup: p2 k= 64 N= 180000 hilos= 2 8.01
Iteraciones por config (mín/máx): 10 10
Salidas incorrectas: 0
```

Figura 13. *Análisis de resultados: iteraciones por configuración, salidas distintas de la serial y mejores speedups.*